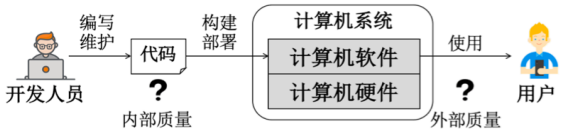
Software Quality



- External quality is the usefulness of the system as perceived from outside. It provides customer value and meets the product owner's specifications.
- This quality can be measured through feature tests, QA and customer feedback.
- Internal quality has to do with the way that the system has been constructed. It affects your ability to manage and reason about the program.
- This quality can be measured through linters, unit tests etc.

外部质量：用户的适用性

内部质量：系统的构建是否易于管理

Code Quality

In addition to **logic correctness**, code quality also concerns

- Understandability (可理解性)
- Maintainability (可维护性)
- Reliability (可靠性)
- Security (安全性)
- Efficiency (高效性)
- Portability (可移植性)

Understandability

Developers spend around 70% of their time understanding code*

- Understandability is the concept that a system should be presented so that an engineer can easily comprehend it.
- The more understandable a system is, the easier it will be for engineers to change it in a predictable and safe manner.

Meaningful Identifier Names

- Use Intention-Revealing Names
- Name Functions as Verbs
- Name Classes as Nouns
- Use Meaningful Distinction
- Use Pronounceable Names
- Use Searchable Names

Meaningful distinction

es

```
int[] arr1;
int[] arr2;
String status;
String statusValue;
```

Noise words like **Data, Value, Info, Variable, Table, String, Object**, etc which are used as a suffix do not offer any meaningful distinction. Noise words are redundant and should be avoided.

Searchable names

```
1  switch (cardState) {
2     case 1: // valid
3        // ...
4        break;
5     case 2: // freezed
6        // ...
7        break;
8     case 3: // expired
9        // ...
10       break;
11    default: ...
12 }
```

Avoid magic numbers: Create named constants instead of using numbers or other constant values where it is supposed to denote something.


```
1  switch (cardState) {
2     case 1: // valid
3        // ...
4        break;
5     case 2: // freezed
6        // ...
7        break;
8     case 3: // expired
9        // ...
10       break;
11    default: ...
12 }
```

Avoid magic numbers: Create named constants instead of using numbers or other constant values where it is supposed to denote something.

Maintainability

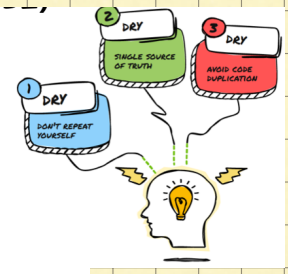
Code maintainability simply means that code is easy to modify, extend, search, or reuse

If your team is doing a good job with code maintainability, the following are true:

- (Search) It's easy for the team to find examples in the codebase, reuse other people's code, and change code maintained by other teams if necessary.
- (Extend) It's easy for the team to add new dependencies to their project, and to migrate to a new version of a dependency.
- (Modify & Reuse) The team's dependencies are stable and rarely break the code.

Code Clones (Duplicate code)

- In a software system, similar or identical fragments of code are known as code clones (or duplicate code)
- Code clones often result from copy/paste programming from within the project or other projects/resources (e.g., online tutorials)



Code clones might be necessary in some cases; however, code clones are generally considered "harmful" in terms of maintainability

- Increase code size
- Increase maintenance cost: If one code fragment contains a bug and gets fixed, all its clones should always be fixed in similar ways

Types of Code Clones

Type 1: Exact copy, only differences in white space and comments.
Type 2: Same as type 1, but also variable renaming.
Type 3: Same as type 2, but also changing or adding few statements.
Type 4: Semantically identical, but not necessarily same syntax.

Initial Code Fragement CF_0

```
for(i = 0; i < 10; i++)
{
    // foo 2
    if (i % 2 == 0)
        a = b + i;
    else
        // foo 1
        a = b - i;
}
```

CF_1 – Type-1 Clone

```
for(i = 0; i < 10; i++)
{
    if (i % 2 == 0)
      a = b + i; //cmt 1
    else
      a = b - i; //cmt 2
}
```

Initial Code Fragement CF_0

```
for(i = 0; i < 10; i++)
{
    // foo 2
    if (i % 2 == 0)
        a = b + i;
    else
        // foo 1
        a = b - i;
}
```

CF_2 – Type-2 Clone

```
for(j = 0; j < 10; j++)
{
    if (j % 2 == 0)
      y = x + j; //cmt 1
    else
      y = x - j; //cmt 2
}
```

Initial Code Fragement CF_0

```
for(i = 0; i < 10; i++)
{
    // foo 2
    if (i % 2 == 0)
        a = b + i;
    else
        // foo 1
        a = b - i;
}
```

CF_3 – Type-3 Clone

```
for(i = 0; i < 10; i++)
{
    // new statement
    a = 10 * b;
    if (i % 2 == 0)
      a = b + i; //cmt 1
    else
      a = b - i; //cmt 2
}
```

Initial Code Fragement CF_0

```
for(i = 0; i < 10; i++)
{
    // foo 2
    if (i % 2 == 0)
        a = b + i;
    else
        // foo 1
        a = b - i;
}
```

CF_4 – Type-4 Clone

```
while(i < 10)
{
    // a comment
  a = (i % 2 == 0) ?
  b + i : b - i;
  i++;
}
```

Reducing Code Clones

Code reuse: turn duplicate code to reusable methods, classes, modules, or libraries

```
int main(int argc, char** argv) {
  int x = 3;
  int y = 2;
  Socket s = opensocket(x, y);
  char buf[80] = readsocket(s, 80);
  closesocket(s);

  int xx = 47;
  int yy = 21;
  Socket ss = opensocket(xx, yy);
  char fub[50] = readsocket(ss, 50);
  closesocket(ss);

  return 0;
}
```

```
void getdata(int x, int y, int size, char * buf) {
  Socket s = opensocket(x, y);
  readsocket(s, buf, 80);
  closesocket(s);
}

int main(int argc, char** argv) {
  char buf[80];
  getdata(3, 2, 80, &buf);

  char fub[50];
  getdata(47, 21, 50, &fub);

  return 0;
}
```

Extract method: Extract duplicate code to a new method

```
public class Context {
  public reload() {
    if (x > y) { System.out.println("Foo!") }
    for (int i = 0; i < modules.length; i++) {
      modules[i].reload();
    }
  }
}

public class FileContext {
  public reload() {
    if (x > y) { System.out.println("Foo!"); }
    for (int i = 0; i < files.length; i++) {
      files[i].reload();
    }
  }
}
```

```
public class AbstractContext {
  public reload() {
    if (x > y) { System.out.println("Foo!") }
    for (int i = 0; i < modules.length; i++) {
      modules[i].reload();
    }
  }
}

public class Context extends AbstractContext {
...
}

public class FileContext extends AbstractContext{
...
}
```

Pull up method: taking a method and "Pulling" it up in the inheritance chain. This is used when a method needs to be used by multiple implementers.

Clone Detection Techniques


```
public class Context {
  public reload() {
    if (x > y) { System.out.println("Foo!") }
    for (int i = 0; i < modules.length; i++) {
      modules[i].reload();
    }
  }
}

public class FileContext {
  public reload() {
    if (x > y) { System.out.println("Foo!"); }
    for (int i = 0; i < files.length; i++) {
      files[i].reload();
    }
  }
}
```

```
public class AbstractContext {
  public reload() {
    if (x > y) { System.out.println("Foo!") }
    for (int i = 0; i < modules.length; i++) {
      modules[i].reload();
    }
  }
}

public class Context extends AbstractContext {
...
}

public class FileContext extends AbstractContext{
...
}
```

Pull up method: taking a method and "Pulling" it up in the inheritance chain. This is used when a method needs to be used by multiple implementers.

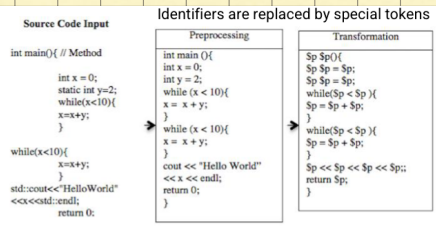
Text Matching

- Older, studied extensively
- Less complex, and most widely used
- No program structure is taken into consideration
- Detect Type-1 clones & some Type-2 clones
- Two types of text matching
  - Exact string match: diff (cvs, svn, git) is based on exact text matching
  - Ambiguous match: LCS, n-gram match

只能检查出Type-1和部分Type-2

Token Sequence Matching

- A little more complex, less widely used
- No program structure is taken into account
- Type-1 and Type-2 clones



http://www.cs.uccs.edu/~jkalita/papers/2016/SheneamerAbdullahIJCA2016.pdf

只将变量重命名了，没有改结构.

能查出Type-1和Type-2

Graph Matching

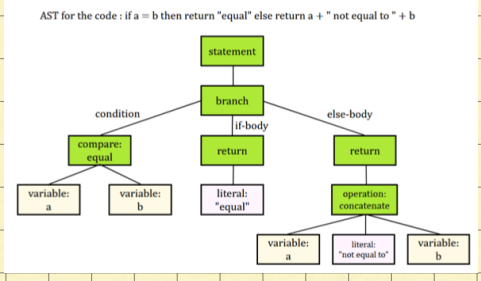
- Newer
- More complex
- Type-1, Type-2, and Type-3 clones

Graphs for modeling source code
- AST (Abstract Syntax Tree)
- CFG (Control Flow Graph)
- PDG (Program Dependency Graph)

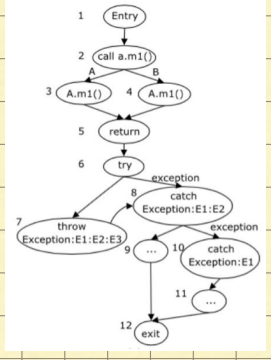
分析了代码结构

能查出Type-1，2，3

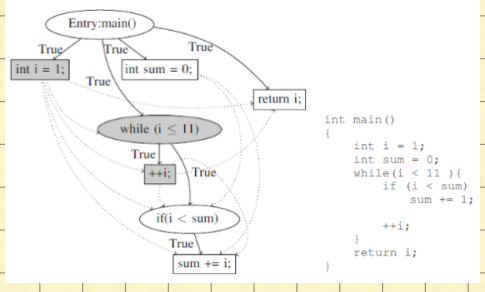
AST



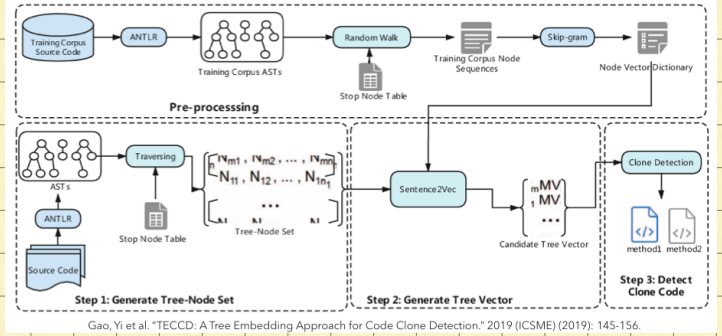
CFG



PDG



(SOTA) Clone Detection Techniques



Gao, Yi et al. "TECCD: A Tree Embedding Approach for Code Clone Detection." 2019 (ICSME) (2019): 145-156.

目前最好的，能查出Type-4.

Reliability and Security

- Reliability is the ability to perform the operations consistently without any failures, every time it runs.
  - In reliability, you're identifying what each chunk of code expects to get, and then define how to handle exceptions, unexpected input.
- Security is the ability to defend against malicious threats (e.g., unauthorized use, information leak, attacks) in order to preserve system privacy, integrity and usability
  - In security, you're looking at the attacks, and making sure your code is secure against them.

Input Validation

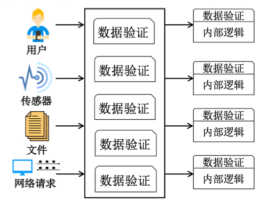
保证正确的数据能进入工作流

- Input validation is performed to ensure only properly formed data is entering the workflow in an information system, preventing malformed data from persisting in the database and triggering malfunction of various downstream components.
- Input validation should happen as early as possible in the data flow, preferably as soon as the data is received from the external party.

越早越好

Untrusted region (outside)
- Out of developers' control
- Potentially malformed or malicious
- Much more restrict requirements for input validation

Barricade: Establish "trust boundaries" — everything outside of the boundary is dangerous, everything inside of the boundary is safe. In the barricade code, validate all input data.



Trusted region (inside)
- Controlled by developers
- Less likely to be exposed to malformed or malicious input
- May be affected by internal defect or expected exceptions

Error Handling

- Error handling refers to the response and recovery procedures from error conditions present in a software application.
- Proper error handling improves software reliability and maintainability
  - Reliability: proper error handling ensures the normal execution of the program
  - Maintainability: proper error handling logs error conditions, which facilitate debugging and fixing.

Strategies of Error Handling

- Maintaining normal service
  - Retry
  - Replacing invalid input with neutral values (e.g., 0), previous input values, or similar valid values
  - Invoking standard error-handling subroutine
- Logging, informing, and terminating
  - Writing error info to log files
  - Return error code or display error message
  - Terminating the program

Choosing Proper Error Handling

- A safety-critical system (SCS) or life-critical system is a system whose failure or malfunction may result in:
  - death or serious injury to people
  - loss or severe damage to equipment/property
  - environmental harm
- For SCS, error handling should value ***software correctness: the software could terminate, but should never return incorrect results.***

软件可以停止，但不能返回不正确结果

- A mission-critical system (MCS) must function continuously in order for a business to be successful.
- If a mission-critical application experiences even brief downtime, the negative consequences are likely to be financial.
- For MCS, error handling should value ***software robustness: the software could tolerate certain errors, but should not terminate***

可以忍受特定错误，但不能停止

Efficiency

- The degree to which the software makes optimal use of system resources
- Efficiency is indicated by the following subattributes:
  - time behavior
  - resource behavior

时间
资源

Portability

- The ease with which the software can be transposed from one environment to another
- Example
  - (Better portability) A Java program that only uses core java APIs
  - (Decreased portability) A Java program that uses native APIs to invoke certain Windows OS services

换环境之后也能运行

Code Review

Code review (sometimes referred to as peer review) is a software quality assurance activity in which one or several people check a program mainly by viewing and reading parts of its source code (changes/commits)

对程序源码的质量审查

Types of code Changes to Be Reviewed

- Modifications to existing code
  - Behavioral changes, improvements, optimizations
  - Bug fixes and rollbacks
- Entirely new code
  - Greenfield code review: review entirely new code
- Automatically generated code
  - E.g., refactoring tools, intelligent code generators

Why Code Review?

- Across the industry, code review is a well recognized, important practice
- Many companies and open-source project have enforced some form of code review
  - Code review is a mandate process in which all software engineers at Google must participate
  - Google engineers spent a large chunk of time in code review
- But code review slows down development and incurs additional cost. So why code review?

A well-designed code review process and a culture of taking code review seriously provides the following benefits

- Code correctness
- Code readability
- Code consistency
- Knowledge sharing
- ……

Code Correctness

- Reviewers typically look for whether a code change
  - was intended
  - was properly designed
  - functions correctly and efficiently
  - has proper testing
  - introduces bugs into the codebase

Many correctness checks are performed automatically through techniques such as static analysis and automated testing

Code Readability

- Code might be written once, but it will be read dozens, hundreds, or even thousands of times
- A code review is often the first test of whether a given code change is understandable to a broader audience

Code Consistency

- Code needs to conform to some standards of consistency so that it can be understood and maintained
- Reviewers should also check
  - Whether the code change follows best practices for the particular programming language
  - Whether the code change is consistent with the particular codebase/repository

Knowledge sharing

- Part of the code review process of feedback and confirmation involves asking questions on why the change is done in a particular way; this exchange of information facilitates knowledge sharing
- Both the authors and reviewers can learn new techniques and patterns from code review
  - Many engineers at Google "meet" other engineers first through their code reviews!

Code Review Flow

- At Google, code reviews take place before a change can be committed to the codebase; this stage is also known as a *precommit review*.
- The primary end goal of a code review is to get another engineer to consent to the change, which is denoted by tagging the change as "looks good to me" (LGTM).
- This LGTM is used as a necessary permissions "bit" to allow the change to be committed.

在代码提交到代码库前需precommit review.

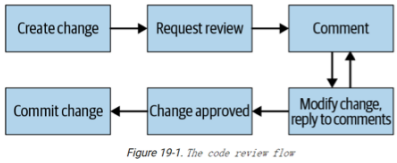
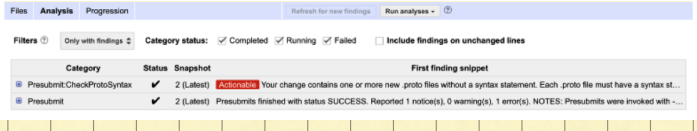


Figure 19-1. The code review flow

Step 1

- A user writes a change to the codebase in their local workspace.
- This *author* then creates a snapshot of the change: a patch and corresponding description that are uploaded to the code review tool (*Critique* at Google).
- This change produces a *diff* against the codebase, which is used to evaluate what code has changed.
  - Uploading a snapshot of the change triggers automated code analyzers (e.g., checking code styles)
  - The author can preview the fix suggestion (e.g., remove the extra spaces) and apply the fix on the change.



Step 2

- When the author is satisfied with the diff of the change, they mail the change to one or more reviewers.
- This process notifies those reviewers, asking them to view and comment on the snapshot.



Step 3

- Reviewers open the change in the code review tool and post comments on the diff.
- Some comments request explicit resolution. Some are merely informational.

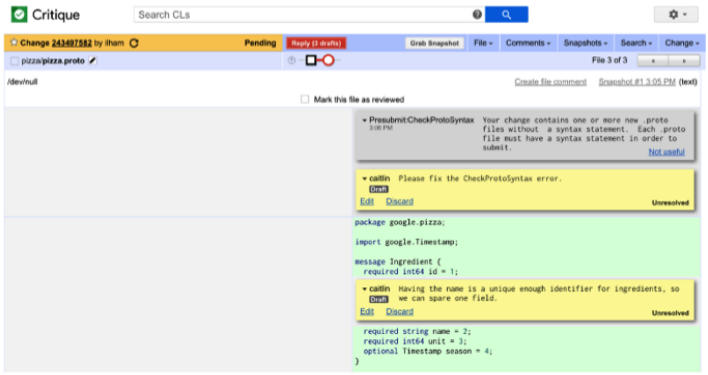


Figure 19-6. Commenting on the diff view

Step 4

- The author modifies the change and uploads new snapshots based on the feedback and then replies back to the reviewers. Steps 3 and 4 may be repeated multiple times.

Step 5

- After the reviewers are happy with the latest state of the change, they agree to the change and accept it by marking it as "looks good to me" (LGTM).
- Only one LGTM is required by default

Step 6

- After a change is marked LGTM, the author is allowed to commit the change to the codebase, provided they resolve all comments and that the change is approved.

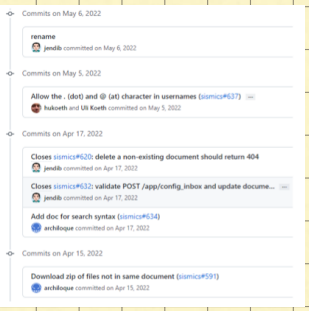
Write Small Changes

- Small changes should generally be limited to ~200 LoC
- Small code changes are easy to digest and tend to focus on a single issue
  - 35% of the changes at Google are to a single file
- Small code changes allow quick approvals and quicker changes to the codebase
  - Most changes at Google are expected to be reviewed within a day

Write Good Change Descriptions

- A change description should
  - summarize the change
  - Explain what is changed and why
  - E.g., “bug fix” is not a good change description

- A good change description also allows Code Search tools to find changes



Code review Best Practices

- Keep reviewers to a minimum
  - Most code reviews at Google involve only 1 reviewer
  - The cost of additional reviewers quickly outweighs their value (the most important LGTM is the first one)

- Automate where possible
  - Rather than require authors to run tests, static analysis tools, or formatters, modern code review tools typically automate these processes
  - The tools detect a variety of problems with the submitted change, reject the change and ask the author to fix the problems (see CI/CD, discussed later)